

MCTR 320 Report

Fall 2025

Isaac Cejas and Totus Tuus Keely

Objective

For our semester-long project, we were tasked with designing a mobile quadruped robot that can travel from one corner of an arena to the opposite, while avoiding obstacles along the path. The project utilizes external sensors connected to a microcontroller/microcomputer to detect obstacles along the path and programming logic to react to the obstacle and change the path.

Approach

Our group purchased a [WAVEGO 12-DOF quadruped robot kit](#) that utilized an ESP32 board, the lower level device, for the calculations required for the movements of the legs, gait control and inverse kinematics, and a Raspberry Pi 5 as the host computer for the robot. Due to the ESP32 having a preprogrammed gait control, we had to only focus on connecting external sensors and creating logic to detect and react to the object and corners as well as defining a path to the corner. This logic could be programmed and run on the Raspberry Pi so the ESP32 did not require any modifications. Originally, we had planned on using a [TFMini-S Micro LiDAR](#) sensor as well as the [Adafruit QMC5883P - Triple Axis Magnetometer](#) as a compass as our external sensors, but these proved difficult to implement through python on the Raspberry Pi. To replace the LiDAR sensor, we attached an [180 degree servo](#) with a PING))) Ultrasonic Distance Sensor mounted on top of the servo. The servo sweeps within a range of angles to scan for objects in the path of the robot. Without the compass, we need to find a new way to determine the right corner. Our robot dog now consists of two paths, the path to the top wall, then the path to the right corner. The robot moves forward until an object is detected, and favors right until the robot is past the object, then turns back forward. The robot will continue this until it has detected the wall as an object and attempts to avoid it but has run the avoid command for long enough time for it to be considered a wall. Once the wall has been detected, the robot initiates its second pathing for the corner.

The robot follows the wall until it detects that the next wall is in front of it, which causes it to stop in the corner.

Coding Logic

Our final execution code consists of standing the robot dog, starting forward movement, then initiating our main function for object detection called **const_sweep** and uses the json, RPi.GPIO, serial and time libraries. Four functions are utilized in this **const_sweep** function. For the external sensors, a **distance()** and **set_angle(angle)** function are defined. The **distance()** function sets the ultrasonic sensor to measure the distance in front of the sensor and return the distance value in centimeters. The **set_angle(angle)** function takes an angle measurement in degrees and converts it to a percentage of the duty cycle by utilizing the equation $\frac{65+angle}{20}$. This duty cycle percentage is then used for PWM of the servo to turn it to the correct angle.

Another function in **const_sweep** is the **Avoid()** function which handles the logic for if an object has been detected within the range of the ultrasonic sensor. The final function utilized throughout the program is the **turn(direction)**. This function takes in a -1 or a 1, which is used to control the JSON code of the robot's gait control. A value of "-1" turns the robot clockwise and "1" turns the robot counterclockwise. Once an object is detected, the robot turns right, records the time and turns the servo orthogonal to the direction of the robot so that it can detect when it passes. Once the robot has passed the object, the robot checks if the object was detected long enough to be considered a wall by subtracting the initial detection time from the final detection time. If a wall has not been detected, the function turns the robot left to be facing the original direction again and returns to the previous **const_sweep** function until it detects another object. If a wall is detected, the function sets the variable **wallDetect** to 1 and returns the variable to the **const_sweep** function.

The main function of **const_sweep** controls the logic for the two paths the robot takes depending on if a wall has been detected. The initial loop of the function and for no wall detected sets the servo to one extreme of the range of angles it sweeps through and increases by 4 degrees each loop while taking a reading of **distance()** with each loop. Once an object has been detected, the loop breaks and runs the previously defined **Avoid()** function. This simple logic loops until the wall has been detected and then activates the second pathing of finding the corner. The second pathing sets the servo to face in line with the robot's gait and walks forward until the corner has been reached. Once the corner has been reached, the code sends a stop command to the robot and the function **const_sweep** breaks.

Project Struggles

Apart from the complexity issues of the LiDAR and compass module causing the switch in detection systems, a couple issues arose during testing of the robot. The major issue was battery life of the robot. We were able to execute a couple trials while troubleshooting sections of the code and values for the object detection but would have issues arise with the ESP32 board outputting errors in the code or with the robot's leg servos not behaving consistently. This led to a longer delay between trials to allow the robot to charge enough for the next trial. Another issue was establishing a connection between the ESP32 and the Raspberry Pi 5. The ESP32 arrived already programmed so we avoided changing the code so that the board would not accidentally break. This left only the Raspberry Pi 5 for us to program the connection to the ESP32. From the company, there were multiple versions of code to establish this connection and required trial and error to find the one that was made for our version of Raspberry Pi. Along with connection to the ESP32, we ran into issues connecting the Raspberry Pi to Wi-Fi. When this issue arose, the easiest option was to reflash the Raspberry Pi and rewrite any code that was lost.